

A Discrete Adjoint Trick for the “I Dropped a Neural Net” Puzzle

Cezar Cocu
me@cezarcocu.com

Dan Sokolsky
dan.sokolsky@gmail.com

Abstract

Jane Street’s “I Dropped a Neural Net” puzzle asks solvers to recover the ordering of 97 shuffled linear layers from a 48-block residual network. Park’s solution pairs each block’s input and output projections, then repairs a rough block ordering with mean-squared-error local search. This note speeds up that repair phase using discrete adjoints. Once a candidate order is close, one forward sweep plus one reverse adjoint sweep gives sensitivities of the output with respect to each intermediate residual state. Those sensitivities score adjacent swaps without re-running the full network for each candidate. On the public puzzle instance, scoring swaps this way recovers the correct permutation from a data-free L1 seed with fewer full-network evaluations than exact adjacent-swap repair.

1 Introduction

Jane Street’s “I Dropped a Neural Net” puzzle provides the weights of a trained residual network after all linear layers have been shuffled, together with 10,000 rows of input data and the original model predictions [4]. The submitted answer is a permutation of the 97 pieces. After shape inspection identifies the scalar head and separates the two projection shapes, the remaining search space is

$$(48!)^2 \approx 10^{122},$$

because each of the 48 input projections must be paired with one of 48 output projections and the recovered blocks must then be ordered.

Park’s arXiv note [6] provided the key structural decomposition: first recover block pairs using the diagonal-dominance structure of $W_{\text{out}}W_{\text{in}}$, then recover the block order with a seed ordering followed by MSE-based adjacent-swap repair. This note keeps that pipeline intact and changes only the repair scoring step: once the pairs are known and the seed order is close, can adjacent swaps be scored without a full forward pass per candidate?

A residual network is naturally a discrete dynamical system. If we cache the forward states and propagate output sensitivities backward through the candidate order, we obtain adjoints for every intermediate state. An adjacent swap changes the state supplied to the unchanged downstream tail after two blocks. The adjoint at that point predicts how the tail will convert that local state change into a prediction change.

This note derives an adjacent-swap score from discrete adjoints, uses it to apply non-overlapping swaps without per-candidate network evaluations, and shows on the public puzzle instance that repair from a data-free L1 seed reaches the solution with fewer full-network evaluations than the MSE bubble-repair baselines we tested. A batched short-insertion extension reuses the same adjoints to reduce repair moves further. Section 4 reports timing against surveyed public CPU solutions.

The implementation repository is available on [GitHub](#) [2].

2 Problem Setup

The shuffled pieces consist of 96 block linear layers plus one scalar head. Those 96 layers form 48 residual blocks: each block has input dimension $d = 48$, hidden dimension $h = 96$, an input projection, and an output projection. The 48 blocks are followed by the scalar head. The provided PyTorch source is equivalent to the following forward structure:

```
class Block(nn.Module):
    def forward(self, x):
        return x + self.out(self.activation(self.inp(x)))

class LastLayer(nn.Module):
    def forward(self, x):
        return self.layer(x)
```

Here **inp**: $\mathbb{R}^{48} \rightarrow \mathbb{R}^{96}$ carries W_{in} , **out**: $\mathbb{R}^{96} \rightarrow \mathbb{R}^{48}$ carries W_{out} , and **layer**: $\mathbb{R}^{48} \rightarrow \mathbb{R}$ is the final linear head. Mathematically, a block k maps $z \in \mathbb{R}^d$ to

$$B_k(z) = z + r_k(z) = z + W_{\text{out}}^{(k)} \text{ReLU}\left(W_{\text{in}}^{(k)} z + b_{\text{in}}^{(k)}\right) + b_{\text{out}}^{(k)}, \quad (1)$$

where $W_{\text{in}}^{(k)} \in \mathbb{R}^{h \times d}$ and $W_{\text{out}}^{(k)} \in \mathbb{R}^{d \times h}$.

For an order $\sigma = (\sigma_0, \dots, \sigma_{47}) \in S_{48}$, define states

$$z_0 = x, \quad z_{t+1} = B_{\sigma_t}(z_t), \quad t = 0, \dots, 47, \quad (2)$$

and prediction

$$f_\sigma(x) = w_{\text{head}}^\top z_{48} + b_{\text{head}}. \quad (3)$$

Each of the 10,000 rows has 48 input columns, the original model prediction **pred**, and a target **true**. We reconstruct against **pred**, written $\hat{y}_i$, not **true**. The objective is

$$L(\sigma) = \frac{1}{N} \sum_{i=1}^N (f_\sigma(x_i) - \hat{y}_i)^2. \quad (4)$$

A recovered solution has $L(\sigma) = 0$ up to floating-point error and emits the expected 97-piece permutation. For reproducibility, our diagnostics record a SHA-256 hash of the emitted permutation string.

As in Park’s solution, we first classify pieces by shape and solve the pairing problem. For each candidate input/output pair, Park computes a diagonal dominance score for $W_{\text{out}} W_{\text{in}}$ and applies the Hungarian algorithm [5]. In our implementation the trace itself is already separated enough on this instance, so we minimize

$$s(i, j) = \text{tr} \left(W_{\text{out}}^{(j)} W_{\text{in}}^{(i)} \right) \quad (5)$$

over one-to-one assignments. This recovers the same 48 block pairs and leaves only the order σ to determine. This raw trace works only because of the sign pattern in this particular model; Park’s diagonal-dominance score is the general method.

3 Adjoint-Predicted Swap Repair

3.1 Data-free seed

The repair phase starts from a seed order. Park uses both a data-dependent delta-norm seed and a data-free $\|W_{\text{out}}\|_F$ seed [6]. Our implementation uses the data-free path, replacing the Frobenius norm, equivalently the L2 norm of the flattened weight matrix, with

$$\|W_{\text{out}}\|_1 = \sum_{a,b} |(W_{\text{out}})_{ab}|. \quad (6)$$

On the public puzzle instance this seed has slightly worse initial MSE than the Frobenius/L2 seed, but needs fewer accepted adjacent swaps under exact bubble repair. Both norms mostly track output-branch magnitude; on this model, the L1 norm may also pick up weight concentration that correlates with depth.

3.2 Discrete adjoints

An adjoint is a sensitivity propagated backward through the network. Here it answers the question: if the residual state at depth t were nudged by a small vector, how would the final scalar prediction change? Computing all such sensitivities takes one backward sweep through the current candidate order, after the usual forward sweep has cached the intermediate states.

For a fixed order σ , the residual network in [equation \(2\)](#) is a discrete dynamical system. For one data row, let

$$J_t = \frac{\partial B_{\sigma_t}}{\partial z}(z_t)$$

be the Jacobian of the block at position t . Away from ReLU kink boundaries,

$$J_t = I + W_{\text{out}}^{(\sigma_t)} D_t W_{\text{in}}^{(\sigma_t)}, \quad D_t = \text{diag}\left(\mathbf{1}\{W_{\text{in}}^{(\sigma_t)} z_t + b_{\text{in}}^{(\sigma_t)} > 0\}\right). \quad (7)$$

Define the tail maps by

$$T_{48}(z) = w_{\text{head}}^\top z + b_{\text{head}}, \quad T_t = T_{t+1} \circ B_{\sigma_t}.$$

Thus T_t maps state z_t to the scalar prediction. Define the adjoint $\lambda_t \in \mathbb{R}^d$ as the sensitivity of this tail output to the intermediate state:

$$\lambda_t = \nabla T_t(z_t).$$

In standard deep-learning terms, λ_t is the gradient of the scalar network output with respect to the intermediate activation z_t . The terminal condition and backward recursion are

$$\lambda_{48} = w_{\text{head}}, \quad \lambda_t = J_t^\top \lambda_{t+1}. \quad (8)$$

This is the same reverse-mode calculation as backpropagation, but we only need gradients with respect to intermediate states, not parameters. For a batch, D_t , J_t , and λ_t are computed row-wise; all formulas with row subscript i use the ReLU mask and adjoint for that row.

Proposition 1 (Local affine tail sensitivity). *Fix an order σ , one input row, and a position t . Let z_j be the forward states generated from z_t through the tail. If every downstream ReLU preactivation in blocks $j = t, \dots, 47$ is nonzero at the reference trajectory, then T_t is affine in some neighborhood of z_t . In that neighborhood,*

$$T_t(z_t + \eta) - T_t(z_t) = \lambda_t^\top \eta.$$

Consequently, $T_t(z_t + \eta) - T_t(z_t) = \lambda_t^\top \eta + o(\|\eta\|)$ as $\eta \rightarrow 0$.

Proof. The nonzero downstream preactivation assumption and continuity imply a neighborhood of z_t in which the downstream ReLU activation pattern is fixed. On that activation cell, every downstream block is affine, so T_t is affine. Applying the chain rule on that cell gives

$$\nabla T_t(z_t) = J_t^\top J_{t+1}^\top \cdots J_{47}^\top w_{\text{head}},$$

which is exactly the backward recursion in [equation \(8\)](#). The local affine identity and the asymptotic expansion follow. $\square$

3.3 Adjacent-swap score

Consider adjacent positions $t, t+1$ with blocks $a = \sigma_t$ and $b = \sigma_{t+1}$. The current two-block state is

$$z_{t+2} = B_b(B_a(z_t)).$$

If the adjacent blocks are swapped, the local two-block state becomes

$$z'_{t+2} = B_a(B_b(z_t)).$$

Computing z'_{t+2} evaluates only the two swapped blocks locally; the downstream tail is not re-run. The local change induced by the swap is

$$\delta_t = z'_{t+2} - z_{t+2}. \quad (9)$$

The downstream network from position $t+2$ onward is unchanged. Using the first-order approximation from [proposition 1](#), the predicted change in scalar output for data row i is

$$\widetilde{\Delta f}_i = \lambda_{t+2,i}^\top \delta_{t,i}. \quad (10)$$

Writing $e_i = f_\sigma(x_i) - \hat{y}_i$, the plug-in predicted MSE change is

$$\widehat{\Delta L} = \frac{1}{N} \sum_{i=1}^N \left(2e_i \widetilde{\Delta f}_i + (\widetilde{\Delta f}_i)^2 \right). \quad (11)$$

The quadratic MSE formula is exact for a given scalar output change. The approximation is using the adjoint estimate of that change.

Remark 1 (Piecewise-linear caveat). *[Equation \(10\)](#) is exact if the original and swapped z_{t+2} states remain in the same downstream ReLU activation cell of the unchanged tail. Local ReLU changes inside the two swapped blocks are already included in δ_t , which is computed exactly. Approximation error enters when the swapped state crosses activation boundaries downstream, or when later accepted swaps make previously computed scores stale. The diagnostics in [section 4](#) quantify this error empirically.*

3.4 Repair loop

Each round performs one forward sweep to cache z_t and one backward sweep to cache λ_t . It then scores all 47 adjacent swaps using [equation \(11\)](#). A screening rule keeps only swaps whose predicted MSE change is negative and large relative to the row-to-row variation of the predicted loss effects. In the implementation, for a candidate swap, we set

$$u_i = 2e_i \widetilde{\Delta f}_i + (\widetilde{\Delta f}_i)^2$$

and compute

$$\hat{\mu} = \frac{1}{N} \sum_i u_i, \quad \hat{\text{se}} = \frac{\text{sd}_{\text{sample}}(u_i)}{\sqrt{N}},$$

and accept a candidate for consideration when

$$\hat{\mu} < -10^{-9} \quad \text{and} \quad \frac{\hat{\mu}}{\hat{\text{se}} + 10^{-12}} < -1.$$

In short, the rule keeps swaps whose predicted improvement is large compared with row-to-row variation on the repair subset. We sort beneficial candidates by predicted improvement and greedily apply a maximal set of non-overlapping swaps, then refresh the forward states and adjoints. Non-overlap avoids applying two swaps to the same block in one round. Each score is still computed from the same pre-round cached trajectory, so accepted swaps upstream or downstream can make other accepted scores stale.

This differs from exact bubble repair. Exact repair evaluates the full network for every candidate adjacent swap. Adjoint repair uses the full network once per round and scores all adjacent candidates with local two-block evaluations plus vector inner products.

3.5 Batched insertion extension

The same local-tail idea also scores short insertion moves. Moving the block at position s to position d changes only the segment $[u, v) = [\min(s, d), \max(s, d) + 1)$ before the unchanged downstream tail begins. Replay that short segment from the cached state z_u using the proposed segment order, yielding z'_v , and set

$$\delta_{s,d} = z'_v - z_v.$$

Then the adjoint at the segment boundary gives the same first-order output change estimate

$$\widetilde{\Delta f}_i = \lambda_{v,i}^\top \delta_{s,d,i},$$

and [equation \(11\)](#) again gives the predicted MSE change. An adjacent swap is the special case $|s - d| = 1$.

The fastest experimental variant scores only a small set of insertions. Each refresh first scores adjacent boundaries, keeps the strongest few, expands those boundaries into insertion proposals within a small window, and greedily applies non-overlapping predicted-improving insertions as one batch. A single exact subset-MSE check after the batch acts as a guardrail: if the batch does not improve the repair subset, it is rejected. This reduces refreshes on this instance; correctness still comes from full-data verification of the reconstructed model.

4 Experiments

4.1 Verification target

All methods target the public puzzle instance. The solver repairs the order using the first 1,000 rows and verifies the reconstructed `nn.Module` on all 10,000 rows. A run is considered hash-matching only when the full-data MSE against the provided `pred` column is below 10^{-9} , the emitted permutation has the expected 97 entries, and the canonical permutation string has SHA-256 hash `093be1cf2d24094db903cbc3e8d33d306ebca49c6accaa264e44b0b675e7d9c4`.

4.2 Public-solver timing context

Solution	Ordering approach	Median full-script wall	Native timer
Ours	L1 seed + adjoint-predicted swaps	1.28 s	0.19 s
alyxya [1]	Greedy order + bubble repair	4.36 s	n/a
Park [7]	Frobenius seed + bubble repair	4.97 s	4.0 s
Hotaj [3]	Cosine pairing + random hill climb	196 s	n/a

Table 1: CPU-pinned timing snapshot for public solvers. All rows were measured as full-script wall time on one Intel Core i9-13900F node with no GPU acceleration. Native timers are solver-reported and are not directly comparable.

Table 1 gives a rough timing scale only; repositories and package versions drift, and not all solutions separate load time from solve time.

4.3 Ablations

Seed	Repair	Seed MSE	Round swaps	Total swaps	Decision work	Final MSE
Frobenius/L2	exact bubble	0.075716	[31, 15, 18, 10, 3, 1]	78	283 exact evals	0
Frobenius/L2	adjoint	0.075716	[13, 8, 11, 9, 6, 6, 4, 4, 3, 3, 2, 3]	72	13 sweeps	0
L1	exact bubble	0.081558	[27, 15, 10, 2]	54	189 exact evals	0
L1	adjoint	0.081558	[13, 8, 8, 7, 8, 9, 5, 3, 1]	62	10 sweeps	0

Table 2: Seed and repair ablations on the public puzzle instance. Decision work counts exact full-network MSE evaluations for exact bubble repair and forward/backward sweeps for adjoint repair. Adjoint diagnostic exact evaluations are excluded.

Table 2 separates the seed choice from the repair method. The exact bubble-repair rows use full-network MSE evaluations for each adjacent candidate. The adjoint rows use full-network candidate evaluations only for diagnostics; those evaluations are not part of the decision rule. Decision work counts high-level decision steps, not FLOPs or wall time.

4.4 Adjoint predictor diagnostics

Seed	Candidates	Pearson r	Sign accuracy	False positives	Accepted
Frobenius/L2	564	0.942	0.977	10	72
L1	423	0.926	0.976	9	62

Table 3: Adjoint-predictor diagnostics. Candidates are adjacent swap scores encountered during adjoint repair; false positives are predicted-improving swaps whose exact subset MSE change is non-negative.

Table 3 compares the predicted MSE change from equation (11) against the exact MSE change for adjacent swap candidates encountered during adjoint repair. The key metric for the repair loop is how often the score predicts the sign of the MSE change among swaps predicted to improve the loss. A false positive means the adjoint score predicted an improving swap, but an exact full-network evaluation would have increased MSE on the repair subset.

4.5 Long-range insertion extension

Repair variant	Moves	Exact checks	Median timer	IQR	Final MSE
Adjacent adjoint	62	0	0.401 s	0.0069 s	0
Batched insertion adjoint	17	4	0.350 s	0.0035 s	0

Table 4: Repair-phase comparison on the same CPU-pinned k8s node over seven runs. The batched insertion variant extends the adjacent adjoint score to short insertion proposals, applies non-overlapping predicted-improving moves in batches, and performs one exact subset-MSE check after each batch. Timers exclude imports, loading, pairing, and final full-data verification.

Table 4 reports a follow-up repair-phase experiment on the same CPU-pinned k8s node used for the public-solver timing snapshot. The batched insertion variant uses the extension described in section 3: adjacent scores identify promising boundaries, short insertion proposals are formed around those boundaries, and non-overlapping predicted-improving insertions are applied with one exact subset-MSE check per batch. It recovered the same final permutation in all seven runs.

The insertion experiment improves the internal repair timer and reduces the number of accepted repair moves on this instance. It also adds more heuristic machinery than adjacent-swap repair.

5 Discussion and Limitations

The adjoint score helps here because adjacent swaps change the activations locally, and cached downstream adjoints often predict the resulting change in the final output. This is the same computational bargain that makes adjoint methods valuable in optimal control and inverse problems: one reverse sweep amortizes many directional sensitivity queries [8].

The limitations are real. First, results are from one public puzzle instance. The L1 seed and adjoint repair loop may rely on properties of this trained model that do not hold in other ResNets. Second, the swap score is first-order; remark 1 describes when it can fail. Third, the algorithm has no global optimality guarantee. It terminates when no adjacent swap passes the repair filter, and the final correctness evidence is empirical: the reconstructed model matches the provided predictions to numerical tolerance and emits the expected 97-piece permutation across the ablations we ran.

In our June 18, 2026 survey of public solutions to this puzzle, including Park [7], alxya [1], and Hotaj [3], we found no prior use of adjoints or vector-Jacobian products to score candidate swaps without a full forward pass.

Overall, this is a small extension of Park’s ordering pipeline. Park identified the structure that makes the puzzle solvable. This note shows that the final MSE repair step can be made cheaper by scoring candidates from cached discrete adjoints.

Declaration of Generative AI Use

The authors used large language model tools, including OpenAI Codex/ChatGPT and Anthropic Claude, to assist with code exploration, implementation, benchmark scripting, prose drafting, copy-editing, and adversarial review. The authors reviewed, edited, and take responsibility for all text, code, mathematical claims, citations, experiments, and conclusions in this note.

References

- [1] alyxya. janestreet-droppedaneuralnet. GitHub repository, 2026. Accessed June 18, 2026. <https://github.com/alyxya/janestreet-droppedaneuralnet>.
- [2] Cezar Cocu. droppedaneuralnet-puzzle. GitHub repository, 2026. Accessed June 18, 2026. <https://github.com/cezarc1/droppedaneuralnet-puzzle>.
- [3] Eugen Hotaj. droppedaneuralnet. GitHub repository, 2026. Accessed June 18, 2026. <https://github.com/EugenHotaj/droppedaneuralnet>.
- [4] Jane Street. I Dropped a Neural Net. Hugging Face Space, 2026. Accessed June 18, 2026. <https://huggingface.co/spaces/jane-street/droppedaneuralnet>.
- [5] Harold W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.
- [6] Hyunwoo Park. I Dropped a Neural Net. [arXiv:2602.19845](https://arxiv.org/abs/2602.19845), 2026.
- [7] Hyunwoo Park. droppedaneuralnet. GitHub repository, 2026. Accessed June 18, 2026. <https://github.com/hynwprk/droppedaneuralnet>.
- [8] Rene-Edouard Plessix. A review of the adjoint-state method for computing the gradient of a functional with geophysical applications. *Geophysical Journal International*, 167(2):495–503, 2006.